

Proyecto Colaborativo MLOps: Predicción de Churn

Curso de MLOps
Profesor: MSaladoM

Fecha límite: **INSERTAR FECHA AQUÍ**

Objetivo

Construir, en equipo de 4 personas, un **pipeline de Machine Learning modular, reproducible y colaborativo** para predecir el abandono de clientes (Churn) en una empresa de telecomunicaciones, simulando un entorno laboral real con roles especializados.

Instrucciones Generales

1. **Formen un equipo de 4 estudiantes** y definan los roles de cada miembro según la tabla de la Sección 3.
2. **Flujo de trabajo con Git (Recomendado):**
 - Un integrante del equipo hace un **"Fork"** del repositorio base desde: <https://github.com/MSaladoM/project-root>.
 - En la configuración del fork, agrega como colaboradores a los otros 3 compañeros y al profesor (MSaladoM).
 - Clonen el fork en sus computadoras locales.
 - Cada miembro crea su propia rama (`feature/data-engineer`, `feature/ml-engineer`, etc.) y trabaja en su módulo asignado.
 - Al finalizar, **el MLOps Engineer del equipo** debe crear un **Pull Request (PR)** desde cada rama hacia la rama `main` del fork y resolver los conflictos.
3. **Alternativa local (si hay problemas con Git):** Si no pueden usar forks/PRs, desarrollen todo el proyecto en una sola computadora de forma colaborativa, pero deben documentar claramente quién hizo cada parte en el informe final. **El flujo con Git es el esperado y el de mayor calificación.**
4. **Uso de LLMs (Large Language Models):**
 - Se **permite y recomienda** el uso de asistentes como ChatGPT, Gemini, Copilot, Llama, etc. para:

- Depurar código.
- Generar estructuras base de funciones.
- Sugerir mejoras en la lógica de preprocesamiento o entrenamiento.
- Redactar fragmentos de pruebas unitarias.
- **Restricciones obligatorias:**
 - a) **No** se permite copiar soluciones completas de otros repositorios sin entenderlas.
 - b) Cada integrante debe poder explicar oralmente cualquier bloque de código generado por un LLM.
 - c) En el archivo `README.md` final, cada rol debe agregar una sección **LLM Contribution** indicando qué herramienta usó y para qué tarea específica (ej. "Usé ChatGPT para generar la estructura del bucle de validación cruzada").
- El uso responsable de LLMs acelera el aprendizaje, pero la evaluación será sobre la integración funcional y la comprensión demostrada.

Dataset

Todos los equipos trabajarán con **Telco Customer Churn** de Kaggle.

- Archivo: `WA_Fn-UseC-Telco-Customer-Churn.csv`
- Problema: Clasificación binaria (Churn = Yes/No)
- **Importante:** Descargar el CSV y guardarlo en `data/raw/`. **NO** subir el CSV a Git (ya está en `.gitignore`).

Roles y Entregables Específicos (Contratos de Interfaz Claros)

Cada módulo debe respetar las siguientes funciones de entrada y salida. Todos deben basarse en `config/params.yaml`.

Rol 1: Data Engineer (`src/data_loader.py`)

Función: `load_and_preprocess_data(config)`

Entrada: diccionario `config` (cargado desde `params.yaml`)

Salida: `X_train`, `X_test`, `y_train`, `y_test` (DataFrames/Series de pandas)

Tareas:

- Cargar CSV desde `data/raw/`.
- Limpiar `TotalCharges`: convertir a numérico, reemplazar espacios vacíos por NaN y luego imputar con mediana.
- Eliminar `customerID`, codificar `gender`, `Partner`, `Churn` a 0/1.
- Dividir usando `test_size` y `random_state` de la configuración.

Rol 2: ML Engineer (src/model_trainer.py)

Función: `train_and_save_model(X_train, y_train, X_test, y_test, config)`

Entrada: datos del Data Engineer + configuración

Salida: diccionario de métricas {accuracy, recall, f1_score}

Tareas:

- Implementar fábrica de modelos: elegir entre RandomForest y LogisticRegression (o SVM) según `config['model_name']`.
- Entrenar el modelo, calcular las 3 métricas.
- Guardar el modelo entrenado con `joblib` en `models/model.pkl`.

Rol 3: MLOps Engineer (src/main.py y config/params.yaml)

Función: `main()` en `main.py` que orquesta el pipeline.

Tareas:

- Crear `params.yaml` con secciones: `data (test_size, random_state)`, `model (model_name, hiperparámetros)`, `paths`.
- En `main.py`, importar y ejecutar las funciones de los roles 1 y 2.
- Asegurar que `python -m src.main` se ejecute sin errores.

Rol 4: QA & Production Engineer (src/predict.py y tests/test_pipeline.py)

Tareas:

- `predict.py`: Script que carga el modelo de `models/model.pkl` y predice sobre un nuevo cliente de ejemplo (puede ser una lista manual). Debe incluir manejo de errores si el modelo no existe.
- `test_pipeline.py`: Al menos dos pruebas unitarias, por ejemplo:
 1. Verificar que `load_and_preprocess_data` no devuelve datos vacíos.
 2. Verificar que `train_and_save_model` devuelve un diccionario con las 3 métricas.

Estructura del Proyecto (ya creada en el fork)

```
churn-mlops-project/  
  config/  
    params.yaml          # Crear y llenar (MLOps)  
  data/  
    raw/                 # Colocar AQUÍ el CSV (NO subir)  
    processed/           # Opcional  
  src/  
    data_loader.py       # Completar (Data Engineer)  
    model_trainer.py     # Completar (ML Engineer)
```

```

    main.py                # Completar (MLOps)
    predict.py             # Completar (QA)
tests/
    test_pipeline.py       # Completar (QA)
models/                   # model.pkl se guarda aquí
requirements.txt          # pandas, scikit-learn, joblib, pyyaml, pytest
.gitignore

```

Checklist de Evaluación

El equipo debe cumplir **todo** lo siguiente:

`python -m src.main` se ejecuta sin errores en la rama `main`.

`config/params.yaml` existe y controla al menos 2 hiperparámetros del modelo.

El ML Engineer permite elegir entre **2 modelos diferentes**.

`predict.py` carga el modelo y hace una predicción de ejemplo.

El historial de Git del fork muestra contribuciones de los 4 miembros.

El `README.md` final incluye Accuracy y Recall del mejor modelo obtenido, más la sección de contribución de LLM.

Entrega

- **Fecha límite:** Jueves 14 de mayo
- **Forma:** Cada equipo comparte la URL de su fork en Google Classroom.
- **Individual:** Cada estudiante escribe en el comentario de la entrega:
 1. El rol que desempeñó.
 2. Una reflexión de 2-3 líneas sobre el mayor desafío de integración que enfrentaron.
 3. Cómo usó un LLM (si lo hizo) para apoyar su trabajo.

Ejemplo de `params.yaml`

```

data:
  test_size: 0.2
  random_state: 42
model:
  name: "RandomForest"
  n_estimators: 100
  max_depth: 10
paths:
  model_save: "models/model.pkl"

```